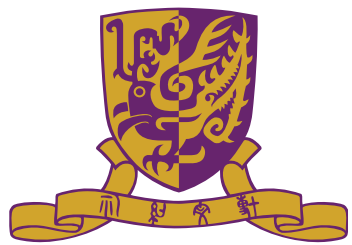
Machine Learning Introduction 4

Transformer

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



"Attention is All You Need"

- Transformer is one special but one the most important architectures of deep learning.
- People come to know it widely because of the following paper:

Attention Is All You Need

Ashish Vaswani[*]
Google Brain
avaswani@google.com

Noam Shazeer[*]
Google Brain
noam@google.com

Niki Parmar[*]
Google Research
nikip@google.com

Jakob Uszkoreit[*]
Google Research
usz@google.com

Llion Jones[*]
Google Research
llion@google.com

Aidan N. Gomez[*] [†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser[*]
Google Brain
lukaszkaiser@google.com

Illia Polosukhin[*] [‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

Transformer is The Foundation of Language Models

"The transformer, today's dominant AI architecture, ...has gone on to revolutionize the field of AI over the past half-decade ..."
— Rob Toews, Contributor. Sep. 3, 2023. Forbes Magazine.

- Contemporary big AI models (language models) mostly use transformer architecture.

Agenda

Attention

Transformer

Attention in Transformer

- Transformer is inherently designed for natural language processing, but now is also widely used for vision tasks.
- The key ingredient of a transformer is attention, which captures the semantic structure of a sentence.

Before introducing the mechanism of attention, we first introduce some basic notions of transformer.

Basic Notions

- The input data to a transformer: $\{\boldsymbol{x}_i\}$. Each $\boldsymbol{x}_i \in \mathbb{R}^d$ is called a token.

Examples:

- For a sentence like

  `It is raining.`

  A token can be a word in this sentence such as "It". It is transformed into a vector $\boldsymbol{x}_i$ by embedding.
- For an image, a token can be a patch.

One can regard $\boldsymbol{x}_i$ as the usual training data point. We call it token instead of training data. Consequently, x_{ij} is the j-th feature of token $\boldsymbol{x}_i$.

Basic Notions

- The input data to a transformer: $\{\boldsymbol{x}_i\}$. Each $\boldsymbol{x}_i \in \mathbb{R}^d$ is called a token.

Examples:

- For a sentence like

  `It is raining.`

  A token can be a word in this sentence such as "It". It is transformed into a vector $\boldsymbol{x}_i$ by embedding.
- For an image, a token can be a patch.

One can regard $\boldsymbol{x}_i$ as the usual training data point. We call it token instead of training data. Consequently, x_{ij} is the j-th feature of token $\boldsymbol{x}_i$.

In transformers, the data matrix $\boldsymbol{X}$ is constructed in a row-wise manner:

$$\boldsymbol{X} = \left[\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\right]^\top \in \mathbb{R}^{n \times d}.$$

Basically, you can regard $\boldsymbol{X}$ as one sentence or one paragraph.

Attention: A Linear Combination

Core idea of attention: Transferring the set of input tokens $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$ to the transformed tokens $\{\boldsymbol{y}_1, \ldots, \boldsymbol{y}_n\}$, so that $\{\boldsymbol{y}_i\}$ contains richer semantic structure.

To perform such a transform, we impose that $\boldsymbol{y}_i$ should not only depend on $\boldsymbol{x}_i$ but the whole set of input tokens $\{\boldsymbol{x}_i\}$ through a linear relationship:

$$\boxed{\boldsymbol{y}_i = \sum_{j=1}^{n} a_{ij} \boldsymbol{x}_j.}$$

Here, $\{a_{ij}\}$ are called attention coefficients. We often impose a simplex structure to $\{a_{ij}\}$, i.e.,

$$a_{ij} \geq 0 \ (\forall i, j), \quad \sum_{j=1}^{n} a_{ij} = 1 \ (\forall i).$$

$\rightsquigarrow \boldsymbol{y}_i$ is calculated by all $\{\boldsymbol{x}_j\}$, and their importance is based on a_{ij}.

Why Attention?

- It is easy to see that attention is to mix all the token information $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$ to generate $\boldsymbol{y}_i$.
- This is because one word in a sentence is not fully meaningful in isolation. We often need to understand the whole semantic meaning, and then understand the meaning of a word (token).

Why Attention?

- It is easy to see that attention is to mix all the token information $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$ to generate $\boldsymbol{y}_i$.
- This is because one word in a sentence is not fully meaningful in isolation. We often need to understand the whole semantic meaning, and then understand the meaning of a word (token).

Let us consider the following two sentences

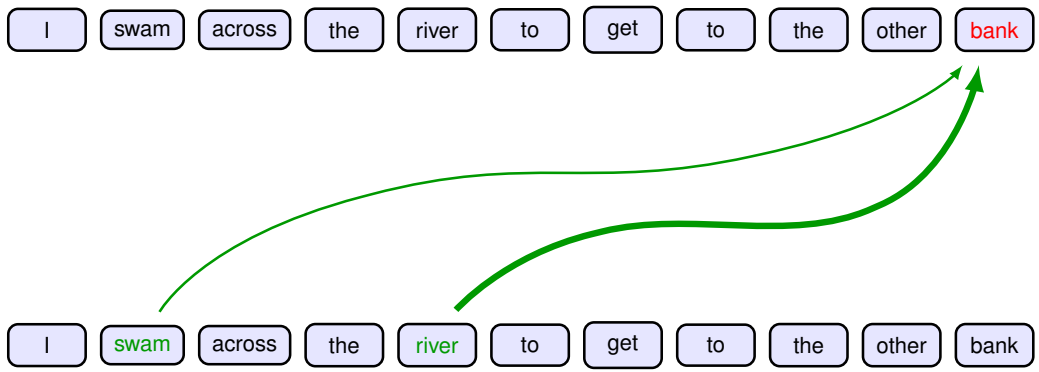
I swam across the river to get to the other bank.
I walked across the road to get cash from the bank.

Suppose the the word "bank" is a token.

- It is the same word "bank" in both two sentences. Do they have the same meaning? Obvious not.
- We need to put the word in the context of the whole sentence, i.e., other words (tokens) also contribute to "bank".

Why Attention?

Hence, attention is to represent the relationship between tokens.



Observations:

- All tokens are related to each other.
- Some tokens are more important to interpret "bank".

$\rightsquigarrow$ Justifies why we need attention mechanism to mix the info of all tokens.

Self-Attention

Self-Attention

However, it is nontrivial to calculate the attention coefficients a_{ij}.

- Since we need to understand the relationship between different tokens.
- We also want to determine the importance of the pairwise relationship. Stronger relation should have more similarity.

Self-Attention

However, it is nontrivial to calculate the attention coefficients a_{ij}.

- Since we need to understand the relationship between different tokens.
- We also want to determine the importance of the pairwise relationship. Stronger relation should have more similarity.

This motivates us to compute the similarity using inner product $\boldsymbol{x}_i^\top \boldsymbol{x}_j$.
Then, the attention coefficients can be constructed using softmax:

$$a_{ij} = \frac{\exp(\boldsymbol{x}_i^\top \boldsymbol{x}_j)}{\sum_{j'=1}^n \exp(\boldsymbol{x}_i^\top \boldsymbol{x}_{j'})}.$$

Such a computed $\{a_{ij}\}$ will satisfy $a_{ij} \geq 0$ $(\forall i, j)$, $\sum_{j=1}^n a_{ij} = 1$ $(\forall i)$.

- If for token $\boldsymbol{x}_i$, we have $a_{ij} > a_{ij'}$ for all other $j' \neq j$, it means $\boldsymbol{x}_{j'}$ is the most important token in the sentence in terms of interpreting token $\boldsymbol{x}_i$.
- This way of determining attention coefficient using tokens themselves is called self-attention.

Self-Attention in Matrix Notation

If we stack the output vectors $\{\boldsymbol{y}_1, \ldots, \boldsymbol{y}_n\}$ to be an output matrix $\boldsymbol{Y}$ as

$$\boldsymbol{Y} = [\boldsymbol{y}_1, \ldots, \boldsymbol{y}_n]^\top.$$

Then, we can have the following matrix form of self-attention

$$\boldsymbol{Y} = \text{softmax}(\boldsymbol{X}\boldsymbol{X}^\top)\boldsymbol{X}.$$

- Here, softmax$(\boldsymbol{L})$ is first taking exponential to all entries of $\boldsymbol{L}$.
- Then, it normalizes each row to sum to one.

Namely,

$$[\text{softmax}(\boldsymbol{X}\boldsymbol{X}^\top)]_{ij} = a_{ij}.$$

Each row of $\boldsymbol{Y}$ is $\boldsymbol{y}_i^\top$. Each row of softmax$(\boldsymbol{X}\boldsymbol{X}^\top)\boldsymbol{X}$ is $\sum_{j=1}^n a_{ij}\boldsymbol{x}_j^\top$.

Trainable Self-Attention

- The attention mechanism is insightful for capturing semantic information. However, the derived self-attention is fixed and has no trainable parameters.

We have three data matrix $\boldsymbol{X}$ in the self-attention formula. Indeed, we can manually add trainable parameters to each data matrix as

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}^{(q)},\ \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}^{(k)},\ \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}^{(v)}.$$

Here, $\boldsymbol{W}^{(q)} \in \mathbb{R}^{d\times d_q}, \boldsymbol{W}^{(k)} \in \mathbb{R}^{d\times d_k}, \boldsymbol{W}^{(v)} \in \mathbb{R}^{d\times d_v}$ are the trainable network parameters.

Trainable Self-Attention

- The attention mechanism is insightful for capturing semantic information. However, the derived self-attention is fixed and has no trainable parameters.

We have three data matrix $\boldsymbol{X}$ in the self-attention formula. Indeed, we can manually add trainable parameters to each data matrix as

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}^{(q)},\ \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}^{(k)},\ \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}^{(v)}.$$

Here, $\boldsymbol{W}^{(q)} \in \mathbb{R}^{d\times d_q}, \boldsymbol{W}^{(k)} \in \mathbb{R}^{d\times d_k}, \boldsymbol{W}^{(v)} \in \mathbb{R}^{d\times d_v}$ are the trainable network parameters.

In this way, the self-attention is generalized to

$$\boldsymbol{Y} = \text{softmax}(\boldsymbol{Q}\boldsymbol{K}^\top)\boldsymbol{V}.$$

In transformer, $\boldsymbol{Q}$ is called query, $\boldsymbol{K}$ is called key, and $\boldsymbol{V}$ is called value. Note that $d_q = d_k$ must be ensured. One typical choice is

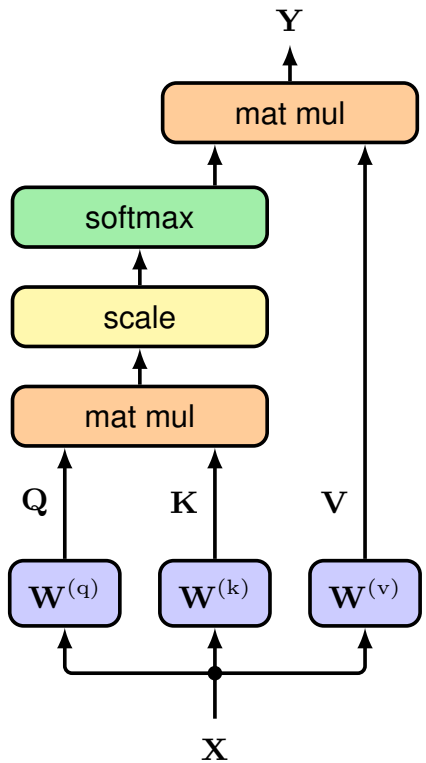
$$d_q = d_k = d_v = d.$$

Scaled Self-Attention

We often re-scale the attention so that gradients of the network will be more stable, giving

$$\boxed{\boldsymbol{Y} = \text{attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}\right)\boldsymbol{V}.}$$

Illustration:



Multi-head Attention

Multi-head Attention

- There might be different patterns of attention. Some patterns may be related to tense, some may be related to vocabulary, etc.
- If we just use one attention, it may learn an averaged information of these multiple patterns.
- Thus, we need multiple attention heads to learn these multiple patterns. Each attention head has its own query, key, value trainable parameters. $\rightsquigarrow$ multi-head attention.

Suppose we have H heads defined as

$$\boldsymbol{H}_h = \text{attention}(\boldsymbol{Q}_h, \boldsymbol{K}_h, \boldsymbol{V}_h), \quad h = 1, \ldots, H.$$

With

$$\boldsymbol{Q}_h = \boldsymbol{X}\boldsymbol{W}_h^{(q)}, \ \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_h^{(k)}, \ \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_h^{(v)}.$$

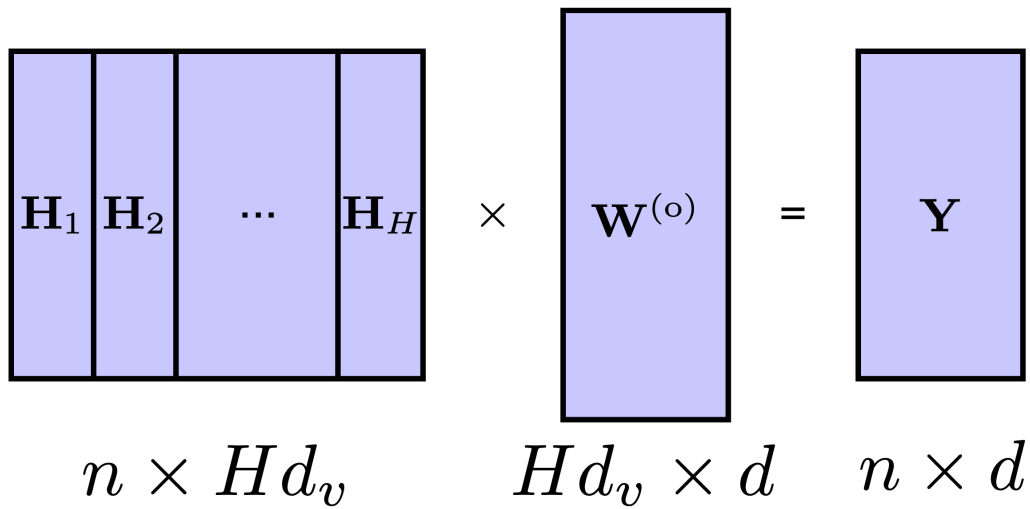
If each $\boldsymbol{W}_h^{(v)}$ has dimension $d \times d_v$, then each $\boldsymbol{H}_h$ has dimension $n \times d_v$.

Multi-head Attention

Then, we can introduce a mixing trainable matrix $\boldsymbol{W}^{(o)}$ to mix all the information as

$$\boxed{\boldsymbol{Y}(\boldsymbol{X}) = [\boldsymbol{H}_1, \ldots, \boldsymbol{H}_H]\boldsymbol{W}^{(o)}.}$$

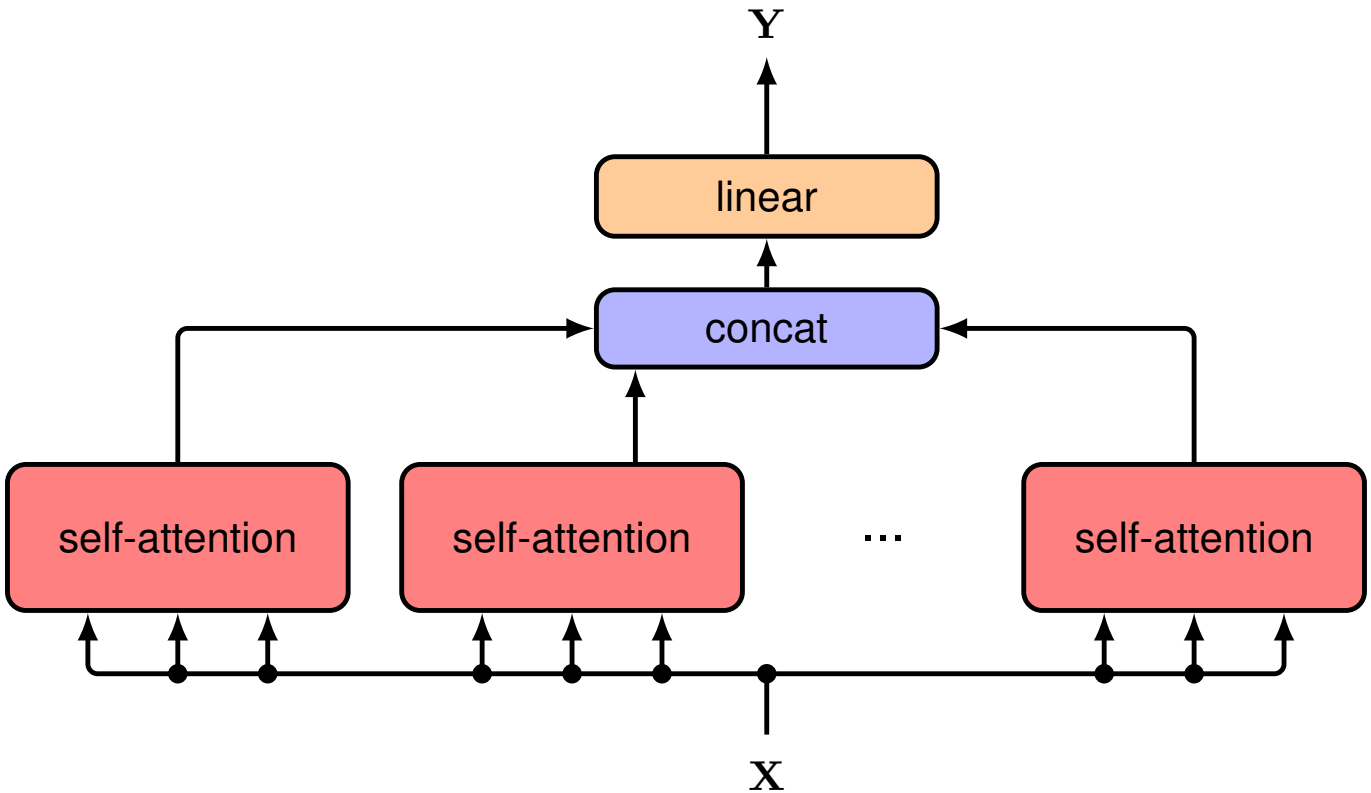
The dimension of the multi-head attention process is illustrated as



Since we want to distribute the total dimension d evenly across all the H heads, we typically choose

$$Hd_v = d.$$

Illustration of Multi-head Attention



Attention Masks

Attention Masks: Motivation

In attention, every token could attend to every other token. However:

- Some positions are invalid (e.g., padding) and must not influence the result.
- Autoregressive decoding must be causal (no information leakage from the future).

Examples:

- Padding sequences with different lengths to the same length (for efficient training):

```
I swam across the river to get to the other bank .
It is raining .  pad pad pad pad pad pad pad pad
```

  The padding tokens are not real signals.

- The second case is causality. Self-attention at position t can attend to the future tokens $\boldsymbol{x}_{>t}$ as we will input the whole sequence. Then, It will minimize loss by copying signals from the future rather than learning real conditional structure.

$\rightsquigarrow$ Attention masks.

Definition of Attention Mask

Masked attention: Introduce a matrix $\boldsymbol{M}$ and re-write attention as

$$\text{attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}; \boldsymbol{M}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}} + \boldsymbol{M}\right)\boldsymbol{V},$$

where

$$\boldsymbol{M}_{ij} = \begin{cases} 0, & \text{allowed} \\ -\infty, & \text{blocked} \end{cases}$$

- An attention mask $\boldsymbol{M} \in \mathbb{R}^{n\times n}$ encodes hard visibility constraints. It is not learned; it is constructed from sequence structure (padding, causality, etc).
- 0 entry means allowed, i.e., allowing i-th token attending to j-th token.
- $-\infty$ simply ensures $\exp(-\infty) = 0$ in softmax.

Example: Padding Mask

Scenario: Variable-length sequences are batched by padding shorter sequences with a PAD token.

- Define a boolean padding indicator vector $\boldsymbol{p} \in \{0,1\}^n$:

$$p_j = \begin{cases} 1, & \text{position } j \text{ is padding} \\ 0, & \text{otherwise} \end{cases}$$

Example: Padding Mask

Scenario: Variable-length sequences are batched by padding shorter sequences with a PAD token.

- Define a boolean padding indicator vector $\boldsymbol{p} \in \{0,1\}^n$:

$$p_j = \begin{cases} 1, & \text{position } j \text{ is padding} \\ 0, & \text{otherwise} \end{cases}$$

- Construct $\boldsymbol{M}^{\text{pad}} \in \mathbb{R}^{n_q \times n_k}$:

$$\boldsymbol{M}^{\text{pad}}_{ij} = \begin{cases} -\infty, & p_j = 1 \\ 0, & p_j = 0 \end{cases} \quad \Longleftrightarrow \quad \boldsymbol{M}^{\text{pad}} = \mathbf{1}\,\boldsymbol{p}^\top \cdot (-\infty),$$

  where $\mathbf{1} \in \mathbb{R}^{n_q}$ is an all-ones vector and $0 \cdot -\infty = 0$.

- Apply:

$$\boldsymbol{Y} = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}} + \boldsymbol{M}^{\text{pad}}\right)\boldsymbol{V}.$$

  This prevents attention to padded keys/values from any query position i.

Example: Causal Mask

Scenario: Maintain causality, token i must not attend to future token $j > i$.

- Define a lower-triangular causal mask $\boldsymbol{M}^{\text{causal}} \in \mathbb{R}^{n \times n}$:

$$\boldsymbol{M}_{ij}^{\text{causal}} = \begin{cases} 0, & j \leq i \\ -\infty, & j > i \end{cases}$$

- Apply (combine with padding mask):

$$\boldsymbol{Y} = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}} + \boldsymbol{M}^{\text{causal}} + \boldsymbol{M}^{\text{pad}}\right)\boldsymbol{V}.$$

Then, all bad attentions will be blocked.

Computational Complexity

Computational Complexity of Attention

Consider the just mentioned masked attention.

- Score matrix: $\boldsymbol{S} = \boldsymbol{Q}\boldsymbol{K}^\top \in \mathbb{R}^{n \times n}$ with cost $O(n^2 d)$.
- Masking + softmax on $\boldsymbol{S}$: Store and softmax $n \times n$ scores with cost $O(n^2)$ (softmax) and memory $O(n^2)$ (score matrix and mask).
- Weighted sum: $\boldsymbol{Y} = \mathrm{softmax}(\boldsymbol{S})\boldsymbol{V}$ with cost $O(n^2 d)$.

Observations:

- Overall, computation and memory are both quadratic in sequence length n.
- Attention is not in linear complexity. One of the major complexity of Transformers.

$\rightsquigarrow$ Next lecture: Transformer architecture using attention mechanism.

Flash-Attention

- Flash-attention is a technique to reduce storage memory from $O(n^2)$ to $O(nd)$, i.e., nearly linear in n when n is very large.

The main idea:

- Rewrites attention as a sequence of smaller block matrix operations.
- Never materializes the full $n \times n$ score matrix in GPU memory.
- Uses an online softmax algorithm to keep numerical stability while processing tiles/blocks.

Complexity of flash-attention:

- Compute: Still $\mathcal{O}(n^2 d)$ (same as standard attention).
- Memory: Reduced from $\mathcal{O}(n^2)$ to roughly $\mathcal{O}(nd)$.

For short sequences (i.e., small n), the benefit is modest. Flash-attention is very useful to reduce GPU memory when n is very large.

Agenda

Attention

Transformer

Transformer Layer

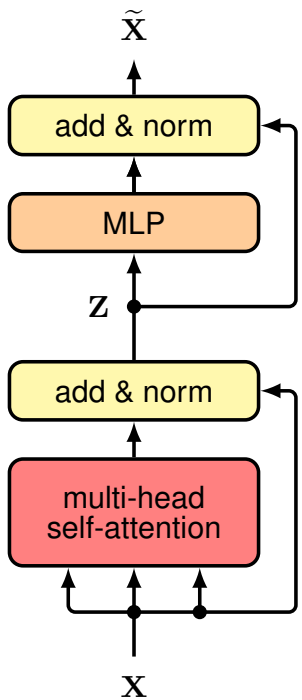
Transformer Layer

- We now need to form a transformer layer.
- We want to add some residual connection to improve efficiency.
- We want to add layer normalization (LayerNorm) for efficiency as well.
- We also want to add several MLP layer (feed-forward layer) to have more flexibility.

Transformer Layer

- We now need to form a transformer layer.
- We want to add some residual connection to improve efficiency.
- We want to add layer normalization (LayerNorm) for efficiency as well.
- We also want to add several MLP layer (feed-forward layer) to have more flexibility.

Overall, a transformer layer has the form:



- The mathematical modeling is:

$$\boldsymbol{Z} = \text{Norm}(\boldsymbol{Y}(\boldsymbol{X}) + \boldsymbol{X}).$$

$$\widetilde{\boldsymbol{X}} = \text{Norm}(\text{MLP}(\boldsymbol{Z}) + \boldsymbol{Z}).$$

- The dimension of the output $\widetilde{\boldsymbol{X}}$ need to be the same as that of the input $\boldsymbol{X}$.

Norm Method I: Layer Normalization (LayerNorm)

Consider $\boldsymbol{x} = [x_1, \ldots, x_d]$ as one data sample, like one row of $\boldsymbol{X}$ (i.e., one token). LayerNorm does the following:

$$\hat{x}_i = \frac{x_i - \mu}{\sigma}, \quad \text{where} \quad \mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2}$$

- The above process is to make the data zero mean and unit variance.

Norm Method I: Layer Normalization (LayerNorm)

Consider $\boldsymbol{x} = [x_1, \ldots, x_d]$ as one data sample, like one row of $\boldsymbol{X}$ (i.e., one token). LayerNorm does the following:

$$\hat{x}_i = \frac{x_i - \mu}{\sigma}, \quad \text{where} \quad \mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2}$$

- The above process is to make the data zero mean and unit variance.

LayerNorm adds two additional trainable parameter vectors $\boldsymbol{\gamma} = [\gamma_1, \ldots, \gamma_d]$ and $\boldsymbol{\beta} = [\beta_1, \ldots, \beta_d]$.
For each dimension $i = 1, \ldots, d$, we then scale (γ_i) and shift (β_i) $\hat{x}_i$ to obtain:

$$y_i = \gamma_i \hat{x}_i + \beta_i.$$

Finally, $\boldsymbol{y} = [y_1, \ldots, y_d]$ will be the layer-normalized data sample.

- LayerNorm can be regarded as giving a new learned mean and variance for each data.
- LayerNorm is important to improving stability of the training process.

Norm Method II: RMSNorm

Another popular norm is Root Mean Square Normalization (RMSNorm).

- First compute the root mean square (RMS)

$$\text{RMS}(\boldsymbol{x}) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \varepsilon}.$$

  where ε is a small number for numerical stability.

- Then, apply normalization using RMS

$$\hat{x}_i = \frac{x_i}{\text{RMS}(\boldsymbol{x})}, \quad y_i = \gamma_i \hat{x}_i.$$

  γ_i is the trainable scaling factor.

Norm Method II: RMSNorm

Another popular norm is Root Mean Square Normalization (RMSNorm).

- First compute the root mean square (RMS)

$$\text{RMS}(\boldsymbol{x}) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \varepsilon}.$$

  where ε is a small number for numerical stability.

- Then, apply normalization using RMS

$$\hat{x}_i = \frac{x_i}{\text{RMS}(\boldsymbol{x})}, \quad y_i = \gamma_i \hat{x}_i.$$

  γ_i is the trainable scaling factor.

Comparison: LayerNorm both scale and shift, while RMSNorm only scales.

- Older / classic LLMs (GPT-2, GPT-3) uses LayerNorm.
- More recent very large LLMs uses RMSNorm.

Post-Norm and Pre-Norm

The method of applying LayerNorm outlined above is called Post-Norm:

$$\begin{aligned} \boldsymbol{Z} &= \text{Norm}(\boldsymbol{Y}(\boldsymbol{X}) + \boldsymbol{X}). \\ \widetilde{\boldsymbol{X}} &= \text{Norm}(\text{MLP}(\boldsymbol{Z}) + \boldsymbol{Z}). \end{aligned}$$

- It post-normalizes output after residual connection (i.e., addition).
- This is the Norm used in the "Attention is All You Need" paper.

However, modern LLMs mainly use pre-norm which pre-normalizes the input:

$$\begin{aligned} \boldsymbol{Z} &= \boldsymbol{Y}(\text{Norm}(\boldsymbol{X})) + \boldsymbol{X}. \\ \widetilde{\boldsymbol{X}} &= \text{MLP}(\text{Norm}(\boldsymbol{Z})) + \boldsymbol{Z}. \end{aligned}$$

As it is found that pre-norm provides a more stable training process.

Complexity of One Transformer Layer

Consider post-norm transformer layer:

$$\boldsymbol{Z} = \text{Norm}(\boldsymbol{Y}(\boldsymbol{X}) + \boldsymbol{X}), \qquad \widetilde{\boldsymbol{X}} = \text{Norm}(\text{MLP}(\boldsymbol{Z}) + \boldsymbol{Z}).$$

Notations. n: sequence length, d: model dimension, d_{MLP}: hidden size of MLP (often $d_{\text{MLP}} \approx 4d$).

- As we studied, the attention module has computation complexity $O(n^2 d)$ and memory complexity $O(n^2)$.

In MLP module (the MLP is often two-layer):

- Computation complexity:

  $O\big(ndd_{\text{MLP}}\big)$ (two linear layers, with elementwise nonlinearity in between)

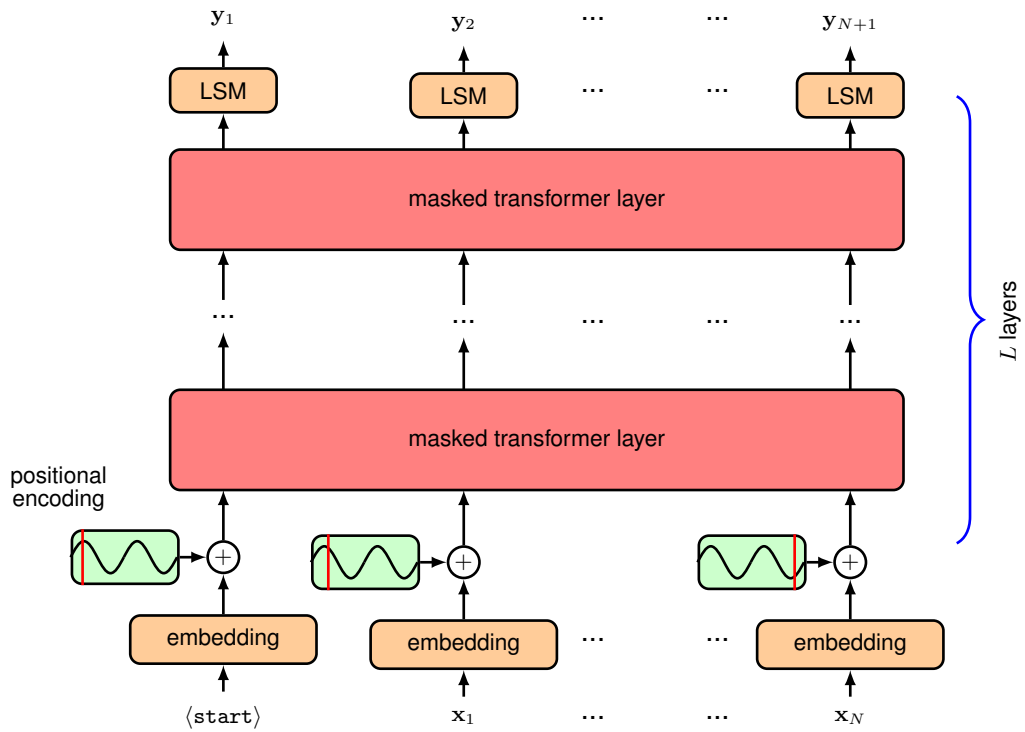
- Memory for activation values:

  $\mathcal{O}\big(nd_{\text{MLP}}\big)$ (need to store d_{MLP} hidden activations in backpropagation)

So the overall order of complexity of the full Transformer layer is quadratic in n.

Transformer Architecture

Decoder Transformer Architecture



- This is decoder transformer, which is the architecture of GPT, Llama, and many other popular language models.
- Transformer is to stack transformer layers multiple times (just like one hidden layer to multiple), and add some other modules.

Tokenization

Goal: Map raw text to a sequence of discrete tokens

$$\text{“It is raining.”} \quad \longrightarrow \quad (t_1, t_2, \ldots, t_n).$$

- Is token in the unit of a character or a whole word? None of them.
  - Characters: very long sequences $\Rightarrow$ inefficient.
  - Words: huge vocabulary, many rare/unknown words.

Tokenization

Goal: Map raw text to a sequence of discrete tokens

$$\text{“It is raining.”} \quad \longrightarrow \quad (t_1, t_2, \ldots, t_n).$$

- Is token in the unit of a character or a whole word? None of them.
  - Characters: very long sequences $\Rightarrow$ inefficient.
  - Words: huge vocabulary, many rare/unknown words.
- Modern LLMs use subword tokenization:
  - Methods: BPE (GPT2), SentencePiece (Qwen3), WordPiece, etc.
  - Common words are single tokens; rare words are split:

$$\text{“unbelievable”} \rightarrow \texttt{``un''}, \texttt{``believ''}, \texttt{``able''}$$

$$\text{“raining”} \rightarrow \text{“rain”}, \text{“ing”}$$

- Each token is represented internally by an integer id

$$t_i \in \{1, \ldots, V\},$$

where V is the vocabulary size (e.g., $V \approx 30\text{k} \sim 100\text{k}$).

Embedding Layer

- The model cannot work directly with token ids t_i; it needs vectors.
- Embedding matrix shared by all embedding positions:

$$\boldsymbol{E} \in \mathbb{R}^{V \times d}, \quad \text{row } \boldsymbol{E}_{t_i:} = \boldsymbol{x}_i^\top \in \mathbb{R}^{1 \times d}.$$

Each token id t_i is mapped to an embedding vector $\boldsymbol{x}_i$:

$$t_i \longrightarrow \boldsymbol{x}_i = \boldsymbol{E}^\top \boldsymbol{e}_{t_i},$$

where $\boldsymbol{e}_{t_i}$ is the one-hot vector of t_i, i.e., taking out the corresponding row of $\boldsymbol{E}$.

Embedding Layer

- The model cannot work directly with token ids t_i; it needs vectors.
- Embedding matrix shared by all embedding positions:

$$\boldsymbol{E} \in \mathbb{R}^{V \times d}, \quad \text{row } \boldsymbol{E}_{t_i:} = \boldsymbol{x}_i^\top \in \mathbb{R}^{1 \times d}.$$

  Each token id t_i is mapped to an embedding vector $\boldsymbol{x}_i$:

$$t_i \longrightarrow \boldsymbol{x}_i = \boldsymbol{E}^\top \boldsymbol{e}_{t_i},$$

  where $\boldsymbol{e}_{t_i}$ is the one-hot vector of t_i, i.e., taking out the corresponding row of $\boldsymbol{E}$.
- Embedding matrix $\boldsymbol{E}$ is learned, rather than designed.
- After adding positional encodings (represent order of words in your sentence), the sequence

$$\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$$

  is fed into the stack of masked transformer layers shown earlier.

The Last Layer: Logistic Regression

- In the last LSM layer, it means linear transformation + softmax, i.e., multi-class logistic regression on the final learned feature $\boldsymbol{Z} \in \mathbb{R}^{(n+1)\times d}$, the added token is `<start>`.
- The number of classes is V, i.e., the vocabulary size. Using classification to predict the next token in the sequence, by choosing a token from the vocabulary.
- The classification is done using logistic regression with lm_head $\boldsymbol{W}_{\text{lm}} \in \mathbb{R}^{d\times V}$, i.e., linear classification weight matrix.

Mathematically,

$$\boldsymbol{P} = \left[\boldsymbol{p}_1, \boldsymbol{p}_2, \ldots, \boldsymbol{p}_{n+1}\right]^\top = \text{softmax}(\boldsymbol{Z}\boldsymbol{W}_{\text{lm}}) = \text{softmax}(\boldsymbol{L}).$$

$\boldsymbol{P}$ is an $(n+1)\times V$ matrix. Every row corresponds to a distribution $\boldsymbol{p}_i$.

The Last Layer: Logistic Regression

- $\boldsymbol{L} = \boldsymbol{Z}\boldsymbol{W}_{\text{lm}} = [\boldsymbol{l}_1, \ldots, \boldsymbol{l}_{n+1}]^\top \in \mathbb{R}^{(n+1)\times V}$ are called logits corresponding to each input token $\boldsymbol{x}_i$.
- $\boldsymbol{p}_i \in \mathbb{R}^V$ is the predicted probability distribution over the vocabulary, for predicting the next token i, as we have shifted the position by adding the starting token `<start>`.
- The groundtruth label for $\boldsymbol{p}_i$ is the real i^{th} token in the sequence/sentence: $\boldsymbol{x}_i$.
- To summarize, the learned feature vector for $\boldsymbol{x}_{i-1}$ (i.e., the i^{th} row of $\boldsymbol{Z}$) captures information of the sentence up to token $\boldsymbol{x}_{i-1}$. We use this vector to obtain $\boldsymbol{p}_i$ and then use it to predict the token $\boldsymbol{x}_i$ (i.e., the label): Next-Token Prediction.

Interpretation: Transformer uses the new attention mechanism. Its overall idea is similar to other neural networks. Trying to extract useful features $\boldsymbol{Z}$ for linear classification.

Tied Embedding and LM Head

Tied parameters in LLMs:

- In decoder-only LMs, the same matrix (or its transpose) is often used for the input embedding and for the LM head matrix $\boldsymbol{W}_{\text{lm}}$ in the final logistic regression layer.

$$\boldsymbol{W}_{\text{lm}} = \boldsymbol{E} \quad \text{(or } \boldsymbol{W}_{\text{lm}} = \boldsymbol{E}^{\top}\text{, depending on convention).}$$

- This is called Tied LM head.
- it reduces parameters by $V \times d$. This is often a very large matrix.

Tied Embedding and LM Head

Tied parameters in LLMs:

- In decoder-only LMs, the same matrix (or its transpose) is often used for the input embedding and for the LM head matrix $\boldsymbol{W}_{\text{lm}}$ in the final logistic regression layer.

$$\boldsymbol{W}_{\text{lm}} = \boldsymbol{E} \quad \text{(or } \boldsymbol{W}_{\text{lm}} = \boldsymbol{E}^{\top}\text{, depending on convention).}$$

- This is called Tied LM head.
- it reduces parameters by $V \times d$. This is often a very large matrix.

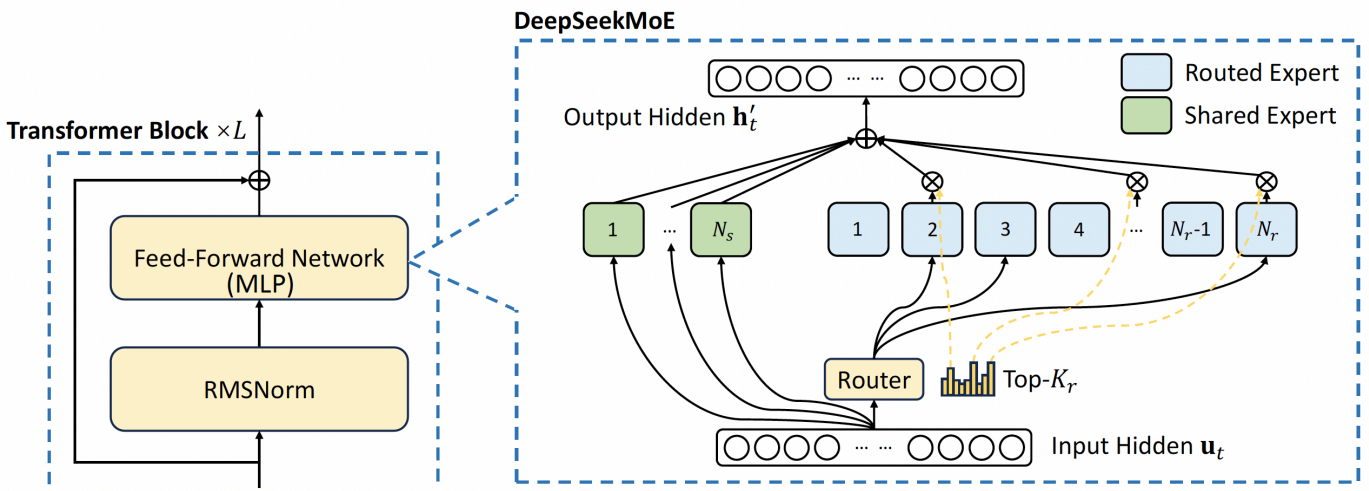
This is reasonable as we use the same embedding to transfer tokens to embedding vectors, and then use the same matrix to transfer embedding (i.e., final hidden state) back to tokens in the vocabulary.

In other words, given the model's final hidden state, we predict the next token as the one whose embedding is most similar (via dot product) to the final hidden state.

Mixture of Experts (MoE)

Mixture-of-Experts (MoE) Feed-Forward Layer

- MoE is to expand the MLP / feed-forward network (FFN), not attention.
- In standard Transformers, each token goes through the same MLP layer.
- In an MoE MLP layer, we have multiple MLPs (experts) and a router that chooses a small subset of experts for each token.
- MoE is a simple architecture scaling approach that can expand the representation power of transformer (said by OpenAI people).



Mathematical Definition of MoE

- Let $\boldsymbol{u}_t \in \mathbb{R}^d$ be the hidden state / activation value of token $\boldsymbol{x}_t$ entering an MoE MLP. The router produces a sparse weighting over N_r experts:

$$\boldsymbol{g}_t = \text{Top-}K_r\big(\text{softmax}(\boldsymbol{R}\boldsymbol{u}_t)\big) \in \mathbb{R}^{N_r},$$

  where $\boldsymbol{R} \in \mathbb{R}^{N_r \times d}$ is the router matrix and K_r is the number of activated experts per token (e.g., $K_r = 2$).
- Only the top-K_r entries of $\boldsymbol{g}_t$ are kept, others are 0, i.e., only the top-K_r routed experts are chosen.

Mathematical Definition of MoE

- Let $\boldsymbol{u}_t \in \mathbb{R}^d$ be the hidden state / activation value of token $\boldsymbol{x}_t$ entering an MoE MLP. The router produces a sparse weighting over N_r experts:

$$\boldsymbol{g}_t = \text{Top-}K_r\big(\text{softmax}(\boldsymbol{R}\boldsymbol{u}_t)\big) \in \mathbb{R}^{N_r},$$

  where $\boldsymbol{R} \in \mathbb{R}^{N_r \times d}$ is the router matrix and K_r is the number of activated experts per token (e.g., $K_r = 2$).
- Only the top-K_r entries of $\boldsymbol{g}_t$ are kept, others are 0, i.e., only the top-K_r routed experts are chosen.

Overall, the MoE layer per token is mathematically defined as

$$\boldsymbol{h}'_t = \underbrace{\boldsymbol{u}_t}_{\text{skip connection}} + \underbrace{\sum_{i=1}^{N_s} \text{MLP}_i^{(s)}(\boldsymbol{u}_t)}_{\text{shared MLP}} + \underbrace{\sum_{i=1}^{N_r} \boldsymbol{g}_t[i] \cdot \text{MLP}_i^{(r)}(\boldsymbol{u}_t)}_{\text{routed expert}}.$$